NATIONAL TEACHERS COLLEGE
School of Arts, Sciences, and Technology — BSIT Program

NTC_CC105 – DATA STRUCTURE W/LAB

WEEK 7 LABORATORY INSTRUCTION

Laboratory Activity 7

Instructor	Prof. Darwin C. Vargas, MIT
Program	BSIT
Academic Year	2026–2027
Programming Language	C++

1. Student Name	Quintela, Nicole M.
2. Student Number	425001249
3. Section	2.4 BSIT
4. Date	Nov. 5, 2026
5. Laboratory Activity No.	Laboratory Activity 7
6. Laboratory Title	Balanced Parentheses and Arithmetic Evaluator in C++

Part 1: Build and verify the stack

Project Setup

- **Folder Name:** CC105_Week8_Section_GroupMeMyselfandI
- **Contains:** GitHub repository link, README.md (inside the GitHub), Code (Stack.h, main.cpp), Folder.

https://github.com/425001249-NTC/Balanced-Parentheses-and-Arithmetic-Evaluator

Implementation Chosen

- Dynamically resized array.
- A reusable template Stack<T>. It is used as Stack<char> (brackets and operators), Stack<double> (evaluation), Stack<size_t> (bracket positions), and Stack<int> (demo).

Operations

- push(value)
- bool pop(T& value)
- bool peek(T& value) const
- bool isEmpty() const
- size() const
- A failed pop or peek returns false and doesn't read invalid memory.

Part 2: Check balanced brackets

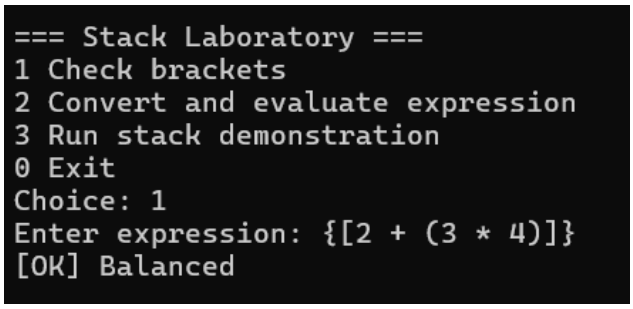
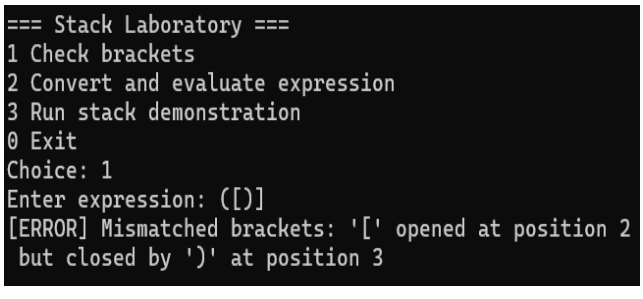
Explanation: Every opening bracket is pushed onto a stack. When a closing bracket comes, we pop the top and check that it matches. The brackets are balanced only if the stack is empty at the end.

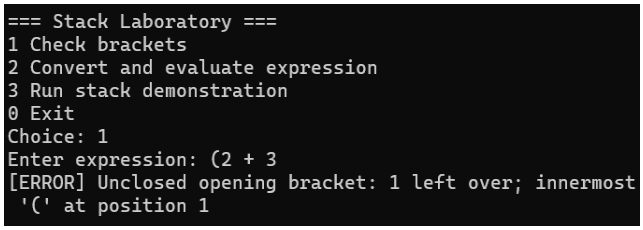
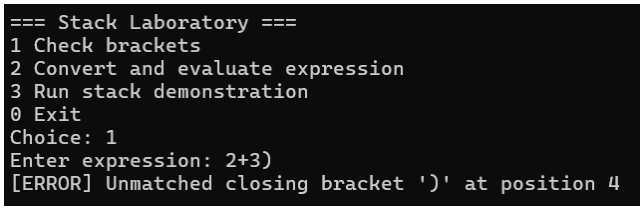
Final Check and Reporting

Three errors are reported clearly:

- Unmatched closing bracket: a closer arrives, and the stack is empty.
- Mismatched pair: the popped opener doesn't match the closer.
- Remaining opening brackets: the stack is not empty at the end.

CheckPoint Test

Input	Result	Program Output
{[2 + (3 * 4)]}	Balance	
([)]	Mismatched Brackets	

(2 + 3	Unclosed opening Bracket	
2 + 3)	Unmatched Closing Bracket	

Checkpoint

- Counting only checks that the number of openers equals the number of closers. ([)] has two of each, but it's still wrong because) arrives when the top of the stack is [. The stack remembers the order, and counting can't.

Part 3: Tokenize and convert to postfix

Explanation: The tokenizer scans left to right, joins digits into one number, skips spaces, and rejects any unsupported character. It also tracks whether the next token must be an operand or an operator, which catches bad order like 2 + * 3. Conversion uses one operator stack. Numbers go straight to the output. (is pushed. An operator first pops operators of greater or equal precedence to the output, then is pushed (* and / are 2, + and - are 1).) Pops to the output until (, then discards the (. At the end, the remaining operators are popped.

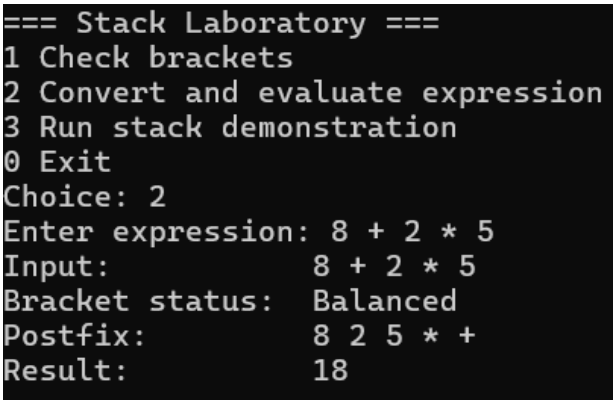
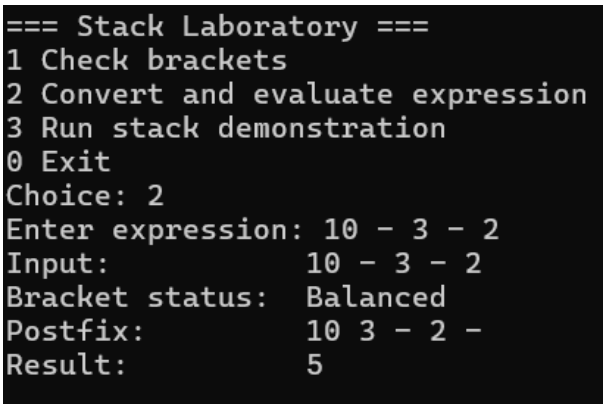
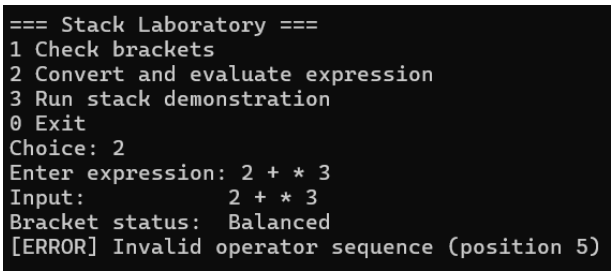
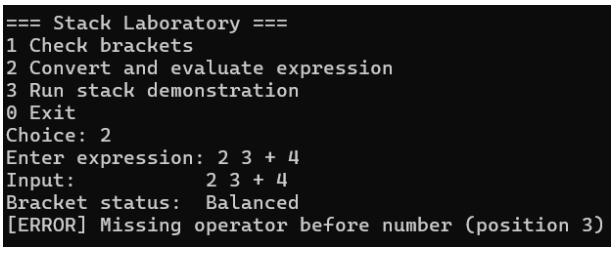
Conversion trace for 2 * (3 + 4) - 5 (operator stack bottom to top)

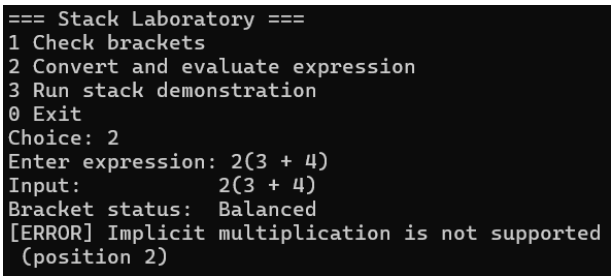
Token	Operator Stack	Postfix Output
2	empty	2
*	*	2
(	* (	2
3	* (	2 3
+	* (+	2 3
4	* (+	2 3 4
)	*	2 3 4 +
-	-	2 3 4 + *
5	-	2 3 4 + * 5
end	empty	2 3 4 + * 5 -

```
=== Stack Laboratory ===
1 Check brackets
2 Convert and evaluate expression
3 Run stack demonstration
0 Exit
Choice: 2
Enter expression: 2 * (3 + 4) - 5
Input:           2 * (3 + 4) - 5
Bracket status:  Balanced
Postfix:         2 3 4 + * 5 -
Result:          9
```

Key Test Result

Input	Expected Output	Program Output

`8 + 2 * 5`	Postfix `8 2 5 * +` (precedence works)	
`10 - 3 - 2`	Postfix `10 3 - 2 -` (left-to-right works)	
`2 + * 3`	Rejected: invalid operator sequence	
`2 3 + 4`	Rejected: missing operator	

`2(3 + 4)`	Rejected: implicit multiplication	

Part 4: Evaluate postfix

Explanation: Postfix evaluation uses one `Stack<double>`. Numbers are pushed. When an operator comes, the program pops the right operand first, then the left operand, computes left operator right, and pushes the result back. It rejects an operator with fewer than two operands, division by zero, extra operands left over, and results that are not finite. After any error, it stops and does not continue evaluating.

Evaluation trace for 2 3 4 + * 5 - (value stack bottom to top)

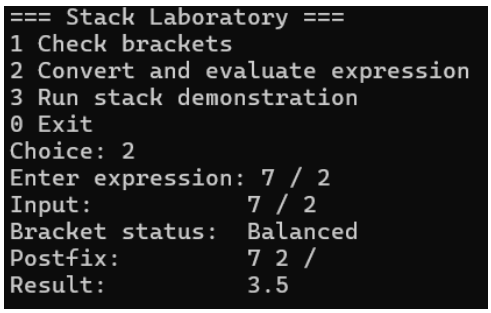
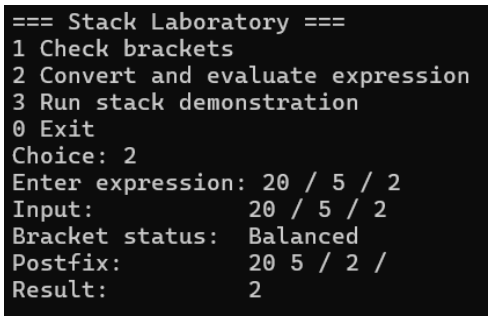
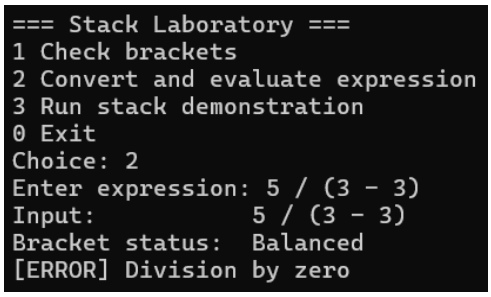
Input	Action	Stack
2	Push	2
3	push	2 3
4	push	2 3 4
+	pop 4 (right), pop 3 (left), 3 + 4 = 7, push	2 7
*	pop 7 (right), pop 2 (left), 2 * 7 = 14, push	14
5	push	14 5
-	pop 5 (right), pop 14 (left), 14 - 5 = 9, push	9

```
=== Stack Laboratory ===
1 Check brackets
2 Convert and evaluate expression
3 Run stack demonstration
0 Exit
Choice: 2
Enter expression: 2 * (3 + 4) - 5
Input:           2 * (3 + 4) - 5
Bracket status:  Balanced
Postfix:         2 3 4 + * 5 -
Result:          9
```

Checkpoint: operand order

- 10 2 - must give 8. The first pop is 2 (right) and the second is 10 (left), so the program calculates 10 - 2. If the order were reversed, it would calculate 2 - 10 = -8, which is wrong. The same rule applies to 20 5 /, which gives 4.

Key test results

Input	Expected Output	Program Output
`7 / 2`	Result 3.5 (doubles are used)	
`20 / 5 / 2`	Result 2	
`5 / (3 - 3)`	Rejected: division by zero	

Part 5: Integrate and test the application
Explaination

- The program shows a repeating menu: 1 Check brackets, 2 Convert and evaluate, 3 Stack demonstration, 0 Exit. After each action, it returns to the menu. An invalid menu choice (like x or 7) shows a message and doesn't crash the program.
- The program shows the input, the bracket status, the postfix expression, and the result. On bad input, it shows a useful error message and returns to the menu, and it never continues evaluating.

Test Case Result

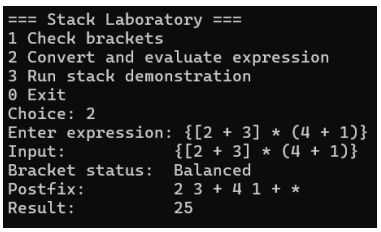
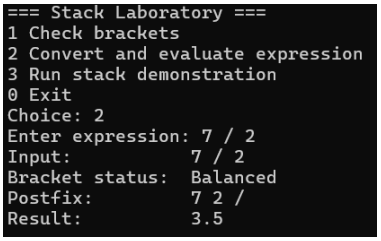
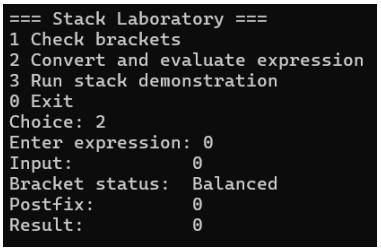
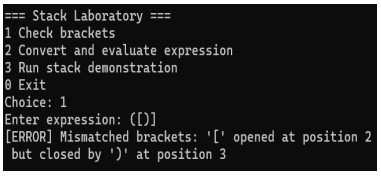
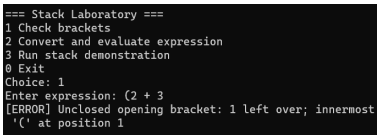
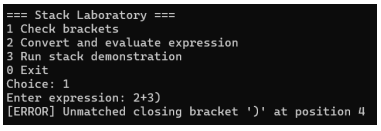
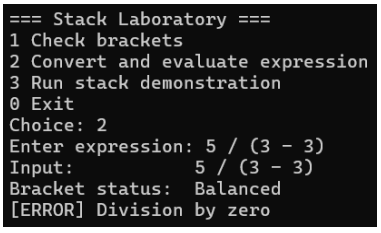
Input Data	Expected Output	Actual Output	Pass/Fail

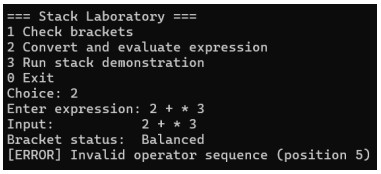
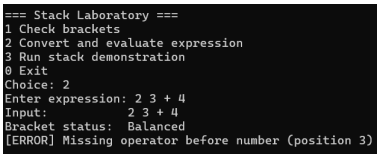
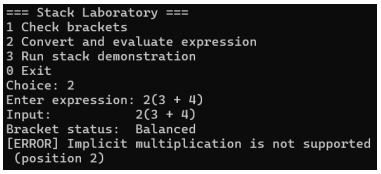
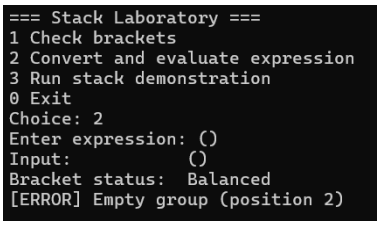
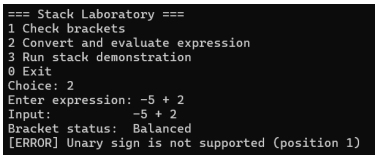
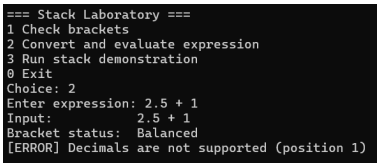
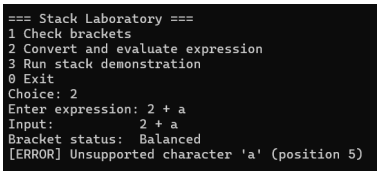
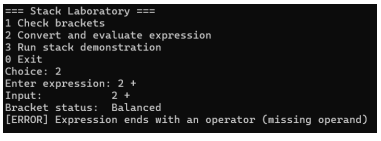
(12 + 3) * 2	12 3 + 2 * ; result 30	`=== Stack Laboratory ===`
		`1 Check brackets`
		`2 Convert and evaluate expression`
		`3 Run stack demonstration`
		`0 Exit`
		`Choice: 2`
		`Enter expression: (12 + 3) * 2`
		`Input: (12 + 3) * 2`
		`Bracket status: Balanced`
		`Postfix: 12 3 + 2 *`
		`Result: 30`
			Pass

8 + 2 * 5	8 + 2 * 5; result 18	`=== Stack Laboratory ===`
		`1 Check brackets`
		`2 Convert and evaluate expression`
		`3 Run stack demonstration`
		`0 Exit`
		`Choice: 2`
		`Enter expression: 8 + 2 * 5`
		`Input: 8 + 2 * 5`
		`Bracket status: Balanced`
		`Postfix: 8 2 5 * +`
		`Result: 18`
			Pass

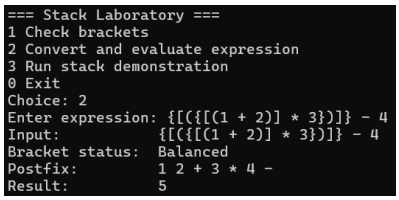
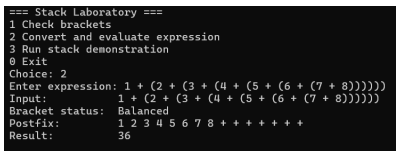
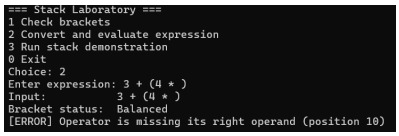
20 / 5 / 2	20 5 / 2 / ; result 2	`=== Stack Laboratory ===`
		`1 Check brackets`
		`2 Convert and evaluate expression`
		`3 Run stack demonstration`
		`0 Exit`
		`Choice: 2`
		`Enter expression: 20 / 5 / 2`
		`Input: 20 / 5 / 2`
		`Bracket status: Balanced`
		`Postfix: 20 5 / 2 /`
		`Result: 2`
			Pass

10 - 3 - 2	10 3 - 2 - ; result 5	`=== Stack Laboratory ===`
		`1 Check brackets`
		`2 Convert and evaluate expression`
		`3 Run stack demonstration`
		`0 Exit`
		`Choice: 2`
		`Enter expression: 10 - 3 - 2`
		`Input: 10 - 3 - 2`
		`Bracket status: Balanced`
		`Postfix: 10 3 - 2 -`
		`Result: 5`
			Pass

{[2 + 3] * (4 + 1)}	2 3 + 4 1 + * ; result 25		**Pass**
7 / 2	7 2 / ; result 3.5		**Pass**
0	0 ; result 0		**Pass**
([)]	mismatched brackets		**Pass**
(2 + 3	Unclosed opening bracket		**Pass**
2 + 3)	Unmatched closing bracket		**Pass**
5 / (3 - 3)	Division by zero		**Pass**

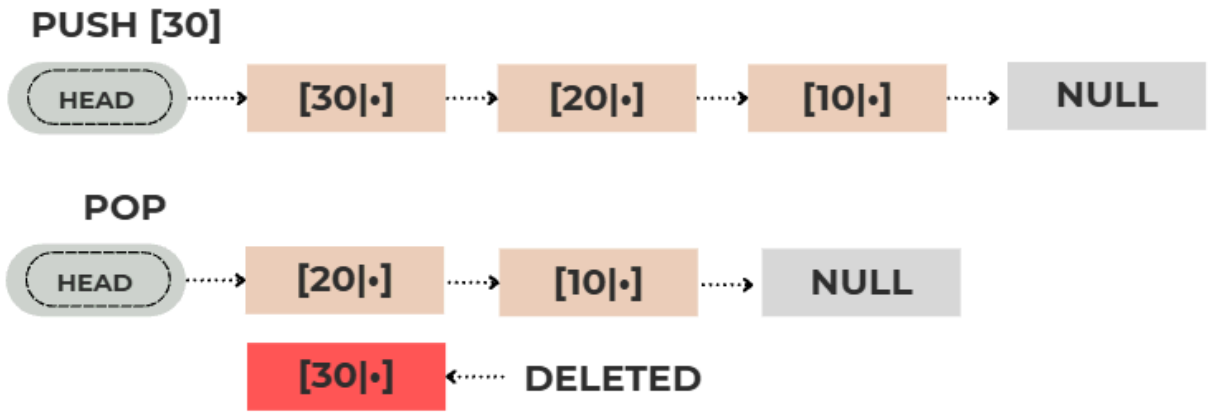
2 + * 3	invalid operator sequence		**Pass**
2 3 + 4	missing operator		**Pass**
2(3 + 4)	implicit multiplication rejected		**Pass**
() or empty input	invalid expression		**Pass**
-5 + 2 or 2.5 + 1	unsupported unary sign or decimal	 	**Pass**
2 + a or 2 +	unsupported character or missing operand	 	**Pass**

Original Test Run

Input Data	Expected Output	Actual Output	Pass/Fail
{[({[(1 + 2)] * 3})]} - 4	1 2 + 3 * 4 −; result 5		**Pass**
1 + (2 + (3 + (4 + (5 + (6 + (7 + 8))))))	1 2 3 4 5 6 7 8 + + + + + + +; result 36		**Pass**
3 + (4 *)	rejected: operator is missing its right operand		**Pass**

Part 6: Compare stack behavior

Sketch of the implementation



Complexity statements

Implementation	push / pop	Note
Linked list	O(1)	Only the head changes.
Dynamic array	push is amortized O(1)	A resize copies everything, which costs O(n), but it happens rarely because the capacity doubles.

Why each expression algorithm is O(n) time and O(n) space (n is the input length)

- **Time:** each algorithm scans the input once. Each character or token gets a constant amount of work, one push, one pop, or one comparison. Every item is pushed at most once and popped at most once, so the total work grows in a straight line with n.
- **Space:** in the worst case, the stacks and the token and postfix lists hold up to n items.

Your stack vs the function call stack

	Function Call Stack	Stack
Memory	Frames are added and removed on each call and return	`new[]` and `delete[]`
Order	LIFO: `helper()` returns first, then `solve()`, then `main()`	LIFO: the last pushed is popped first

Reflection

19. Why does LIFO correctly match nested brackets?

- The last bracket you open is the first one you must close. A stack works the same way. The last item you put in is the first one that comes out. So when a closing bracket comes, I check the top of the stack. If it matches, it's correct. If it doesn't, it's wrong. Just counting brackets can't catch ([)], but the stack can.

20. Why must evaluation pop the right operand before the left operand?

- The right number is pushed last, so it's on top and comes out first. Order matters for subtraction and division. For 10 2 -, the first pop is 2 (right) and the second is 10 (left), so I do 10 - 2 = 8. If I reversed it, we'd get 2 - 10 = -8, which is wrong.

21. What is the difference between balanced brackets and a valid arithmetic expression?

- Balanced brackets means every bracket has a matching partner. A valid expression also needs numbers and operators in the right order.

22. Compare your stack ADT with the function call stack. In a nested function call, which call returns first?

- Both are LIFO, meaning the last one in is the first one out. The function call stack is handled by the computer automatically. The stack is handled by my own code, where I write push and pop and manage the memory myself. If main() calls solve() and solve() calls helper(), then helper() returns first because it was called last.

23. What memory is allocated and released by your implementation? Explain one resizing or node-deletion step.

- My stack uses an array in heap memory. It starts with space for 2 items. When it's full, I make a new array twice as big, copy the old items into it, and delete the old array. For example, if the stack has 10 and 20 and is full, pushing 30 makes a new array of size 4, copies 10 and 20, deletes the old array, and then adds 30. When the stack is no longer needed, the destructor deletes the array.

ScreenShots

```
=== Stack Laboratory ===
1 Check brackets
2 Convert and evaluate expression
3 Run stack demonstration
0 Exit
Choice: 2
Enter expression: 10 - 3 - 2
Input:           10 - 3 - 2
Bracket status:  Balanced
Postfix:         10 3 - 2 -
Result:          5
```

```
=== Stack Laboratory ===
1 Check brackets
2 Convert and evaluate expression
3 Run stack demonstration
0 Exit
Choice: 2
Enter expression: (12 + 3) * 2
Input:           (12 + 3) * 2
Bracket status:  Balanced
Postfix:         12 3 + 2 *
Result:          30
```

```
=== Stack Laboratory ===
1 Check brackets
2 Convert and evaluate expression
3 Run stack demonstration
0 Exit
Choice: 2
Enter expression: 8 + 2 * 5
Input:           8 + 2 * 5
Bracket status:  Balanced
Postfix:         8 2 5 * +
Result:          18
```

```
=== Stack Laboratory ===
1 Check brackets
2 Convert and evaluate expression
3 Run stack demonstration
0 Exit
Choice: 2
Enter expression: 20 / 5 / 2
Input:           20 / 5 / 2
Bracket status:  Balanced
Postfix:         20 5 / 2 /
Result:          2
```

```
=== Stack Laboratory ===
1 Check brackets
2 Convert and evaluate expression
3 Run stack demonstration
0 Exit
Choice: 2
Enter expression: {[2 + 3] * (4 + 1)}
Input:           {[2 + 3] * (4 + 1)}
Bracket status:  Balanced
Postfix:         2 3 + 4 1 + *
Result:          25
```

```
=== Stack Laboratory ===
1 Check brackets
2 Convert and evaluate expression
3 Run stack demonstration
0 Exit
Choice: 2
Enter expression: 7 / 2
Input:           7 / 2
Bracket status:  Balanced
Postfix:         7 2 /
Result:          3.5
```

```
=== Stack Laboratory ===
1 Check brackets
2 Convert and evaluate expression
3 Run stack demonstration
0 Exit
Choice: 2
Enter expression: 0
Input:           0
Bracket status:  Balanced
Postfix:         0
Result:          0
```

```
=== Stack Laboratory ===
1 Check brackets
2 Convert and evaluate expression
3 Run stack demonstration
0 Exit
Choice: 1
Enter expression: ([])
[ERROR] Mismatched brackets: '[' opened at position 2 but closed by ')' at position 3
```

```
=== Stack Laboratory ===
1 Check brackets
2 Convert and evaluate expression
3 Run stack demonstration
0 Exit
Choice: 1
Enter expression: (2 + 3
[ERROR] Unclosed opening bracket: 1 left over; innermost '(' at position 1
```

```
=== Stack Laboratory ===
1 Check brackets
2 Convert and evaluate expression
3 Run stack demonstration
0 Exit
Choice: 2
Enter expression: 5 / (3 - 3)
Input:           5 / (3 - 3)
Bracket status:  Balanced
[ERROR] Division by zero
```

```
=== Stack Laboratory ===
1 Check brackets
2 Convert and evaluate expression
3 Run stack demonstration
0 Exit
Choice: 2
Enter expression: 2 + * 3
Input:           2 + * 3
Bracket status:  Balanced
[ERROR] Invalid operator sequence (position 5)
```

```
=== Stack Laboratory ===
1 Check brackets
2 Convert and evaluate expression
3 Run stack demonstration
0 Exit
Choice: 1
Enter expression: 2+3)
[ERROR] Unmatched closing bracket ')' at position 4
```

```
=== Stack Laboratory ===
1 Check brackets
2 Convert and evaluate expression
3 Run stack demonstration
0 Exit
Choice: 2
Enter expression: ()
Input:           ()
Bracket status:  Balanced
[ERROR] Empty group (position 2)
```

```
=== Stack Laboratory ===
1 Check brackets
2 Convert and evaluate expression
3 Run stack demonstration
0 Exit
Choice: 2
Enter expression: 2 3 + 4
Input:           2 3 + 4
Bracket status:  Balanced
[ERROR] Missing operator before number (position 3)
```

```
=== Stack Laboratory ===
1 Check brackets
2 Convert and evaluate expression
3 Run stack demonstration
0 Exit
Choice: 2
Enter expression: 2(3 + 4)
Input:           2(3 + 4)
Bracket status:  Balanced
[ERROR] Implicit multiplication is not supported (position 2)
```

```
=== Stack Laboratory ===
1 Check brackets
2 Convert and evaluate expression
3 Run stack demonstration
0 Exit
Choice: 2
Enter expression: 2 + a
Input:           2 + a
Bracket status:  Balanced
[ERROR] Unsupported character 'a' (position 5)
```

```
=== Stack Laboratory ===
1 Check brackets
2 Convert and evaluate expression
3 Run stack demonstration
0 Exit
Choice: 2
Enter expression: -5 + 2
Input:           -5 + 2
Bracket status:  Balanced
[ERROR] Unary sign is not supported (position 1)
```

```
=== Stack Laboratory ===
1 Check brackets
2 Convert and evaluate expression
3 Run stack demonstration
0 Exit
Choice: 2
Enter expression: 2.5 + 1
Input:           2.5 + 1
Bracket status:  Balanced
[ERROR] Decimals are not supported (position 1)
```

```
=== Stack Laboratory ===
1 Check brackets
2 Convert and evaluate expression
3 Run stack demonstration
0 Exit
Choice: 2
Enter expression: 2 +
Input:           2 +
Bracket status:  Balanced
[ERROR] Expression ends with an operator (missing operand)
```

```
=== Stack Laboratory ===
1 Check brackets
2 Convert and evaluate expression
3 Run stack demonstration
0 Exit
Choice: 2
Enter expression: 1 + (2 + (3 + (4 + (5 + (6 + (7 + 8))))))
Input:          1 + (2 + (3 + (4 + (5 + (6 + (7 + 8))))))
Bracket status:  Balanced
Postfix:        1 2 3 4 5 6 7 8 + + + + + + +
Result:         36
```

```
=== Stack Laboratory ===
1 Check brackets
2 Convert and evaluate expression
3 Run stack demonstration
0 Exit
Choice: 2
Enter expression: 2 * (3 + 4) - 5
Input:          2 * (3 + 4) - 5
Bracket status:  Balanced
Postfix:        2 3 4 + * 5 -
Result:         9
```

```
=== Stack Laboratory ===
1 Check brackets
2 Convert and evaluate expression
3 Run stack demonstration
0 Exit
Choice: 2
Enter expression: 3 + (4 * )
Input:          3 + (4 * )
Bracket status:  Balanced
[ERROR] Operator is missing its right operand (position 10)
```


```
=== Stack Laboratory ===
1 Check brackets
2 Convert and evaluate expression
3 Run stack demonstration
0 Exit
Choice: 2
Enter expression: 3 + (4 * )
Input:          3 + (4 * )
Bracket status:  Balanced
[ERROR] Operator is missing its right operand (position 10)
```


```
=== Stack Laboratory ===
1 Check brackets
2 Convert and evaluate expression
3 Run stack demonstration
0 Exit
Choice: 3
  (start) -> - | contents [] | size 0 | capacity 2
  pop on empty -> false | contents [] | size 0 | capacity 2
  push 10 -> - | contents [10] | size 1 | capacity 2
  push 20 -> - | contents [10, 20] | size 2 | capacity 2
  peek -> 20 | contents [10, 20] | size 2 | capacity 2
  push 30 (array grows) -> - | contents [10, 20, 30] | size 3 | capacity 4
  pop -> 30 | contents [10, 20] | size 2 | capacity 4
  pop -> 20 | contents [10] | size 1 | capacity 4
  pop -> 10 | contents [] | size 0 | capacity 4
  peek on empty -> false | contents [] | size 0 | capacity 4
```